STAYMAP POLSKA

DOKUMENTACJA KOŃCOWA PROJEKTU

Wersja dokumentu:	1.0
Data opracowania:	Kwiecień 2026
Przeznaczenie:	Zarząd i interesariusze — przegląd architektury, zakresu i eksploatacji
Status:	Dokumentacja końcowa — projekt zrealizowany
Środowisko:	Monorepo: backend Django 5 + frontend Next.js 14

Dokument sporządzony na podstawie analizy repozytorium i konfiguracji systemu (stan: kwiecień 2026).

Przeznaczony wyłącznie do użytku wewnętrznego. Wszelkie prawa zastrzeżone.

. Spis treści

1. Streszczenie wykonawcze 3

2. Cel produktu i grupy użytkowników 4

3. Architektura systemu 5

3.1 Widok ogólny — warstwy systemu 5

3.2 Przepływ żądania REST 6

3.3 Komunikacja w czasie rzeczywistym (WebSocket) 6

3.4 Zadania asynchroniczne (Celery) 7

4. Stack technologiczny 8

4.1 Backend 8

4.2 Frontend 9

4.3 Infrastruktura i narzędzia 9

5. Moduły domenowe backendu 10

6. Funkcjonalności — opis dla zarządu 14

6.1 Konta i tożsamość użytkowników 14

6.2 Oferty i wyszukiwanie 15

6.3 Silnik cen i rezerwacje 16

6.4 Panel gospodarza 18

6.5 Moderacja treści 19

6.6 System recenzji (z mechanizmem ślepego ujawnienia) 19

6.7 Komunikacja i czat w czasie rzeczywistym 20

6.8 Asystent AI — wyszukiwanie w języku naturalnym 21

6.9 Odkrywanie ofert i porównywarka 22

6.10 Inteligencja lokalizacji (POI i scoring) 23

7. Frontend — mapa aplikacji 24

8. API — przegląd endpointów 26

9. Bezpieczeństwo 28

10. Operacje i wdrożenie produkcyjne 29

11. Testy i CI/CD 31

12. Zależności zewnętrzne 32

13. Diagram przepływu rezerwacji 33

14. Otwarte tematy i dalszy rozwój 34

15. Słowniczek pojęć 35

1. Streszczenie wykonawcze

StayMap Polska to platforma rezerwacji noclegów w Polsce, zaprojektowana z myślą o użytkowniku ceniącym wygodę i przejrzystość. Centralnym punktem aplikacji jest interaktywna mapa, na której gość odnajduje i porównuje oferty — domki, glamping, pensjonaty, agroturystykę — zanim zdecyduje się na rezerwację. Po stronie gospodarza platforma dostarcza kompletny zestaw narzędzi: zarządzanie ofertą, elastyczny cennik z obsługą polskiego kalendarza świąt, kalendarz dostępności, komunikację z gościem oraz wgląd w statystyki.

Od strony technicznej system jest zbudowany jako **aplikacja dwuwarstwowa**: backend REST API oparty na **Django 5 + PostGIS** oraz nowoczesny frontend w **Next.js 14**. Zadania wymagające pracy w tle — wysyłka e-maili, anulowanie nieaktywnych rezerwacji, odświeżanie cache POI — obsługuje **Celery** z brokerem **Redis**. Czat między gościem a gospodarzem działa w czasie rzeczywistym dzięki **WebSocketom** (Django Channels + Daphne). Wyszukiwanie w języku naturalnym realizuje moduł AI oparty na **OpenAI API**.

Obszar	Opis
Kluczowa wartość	Jeden spójny produkt łączący wyszukiwanie geograficzne, dynamiczny cennik, profil hosta i gościa, recenzje, czat oraz moduły AI
Docelowy rynek	Polska — noclegi turystyczne (góry, morze, jeziora, wellness, agroturystyka)
Użytkownicy	Goście (poszukujący noclegu), Gospodarze (właściciele obiektów), Administratorzy (moderacja)
Stan projektu	Zrealizowany — wszystkie funkcjonalności i widoki gotowe, infrastruktura CI/CD skonfigurowana
Gotowość produkcyjna	Backend i frontend gotowe do wdrożenia; płatności Stripe przygotowane infrastrukturalnie, wymagają konfiguracji środowiskowej

Kluczowe wyróżniki platformy:

- **Map-first UX** — wyszukiwanie na mapie z klasteryzacją markerów (Leaflet)
- **Silnik cen** uwzględniający sezonowość, polskie święta, dopłaty za dodatkowych gości i długi pobyt
- **Asystent AI** rozumiejący zapytania w języku naturalnym (np. 'domek dla 4 osób na Mazurach w sierpniu z sauną')
- **Czat w czasie rzeczywistym** między gościem a gospodarzem (WebSocket)
- **Moderacja** zapewniająca jakość treści przed publikacją oferty
- **Mechanizm ślepego ujawnienia recenzji** — obie strony oceniają się niezależnie przed upublicznieniem
- **Inteligencja lokalizacji** — automatyczne podsumowanie POI w okolicy każdej oferty
- **Porównywarka ofert** z sesją dla gościa anonimowego i zalogowanego

2. Cel produktu i grupy użytkowników

Platforma odpowiada na konkretną lukę rynkową: brak w Polsce dedykowanego narzędzia do odkrywania noclegów turystycznych z mapą jako głównym interfejsem, dynamicznym cennikiem i wbudowaną komunikacją. StayMap Polska integruje perspektywę trzech grup użytkowników w jednym systemie.

Grupa	Potrzeby	Obsługa w systemie
Gość (podróżny)	Znalezienie odpowiedniego noclegu, sprawdzenie ceny i dostępności, szybka rezerwacja, zarządzanie pobytami, komunikacja z gospodarzem	Wyszukiwanie na mapie i filtry, karta oferty, widget rezerwacyjny z wyceną, konto z historią pobytów, lista życzeń, czat, recenzje, porównywarka, asystent AI
Gospodarz (właściciel obiektu)	Publikacja i promocja oferty, zarządzanie kalendarzem i cennikiem, obsługa gości, kontrola jakości rezerwacji	Panel hosta, onboarding, CRUD ofert + upload zdjęć, reguły cenowe, kalendarz blokad, kolejka rezerwacji, szablony wiadomości, recenzje hosta
Administrator (moderator)	Zapewnienie jakości i bezpieczeństwa publikowanych treści, weryfikacja ofert	Kolejka ofert oczekujących, akcje zatwierdź/odrzuć z komentarzem, dostęp do panelu administracyjnego Django

Założenia produktowe

System nie jest globalnym agregatorem — skupia się wyłącznie na Polsce i polskim rynku noclegowym, co przekłada się na konkretne decyzje projektowe: polska lokalizacja interfejsu, obsługa polskich świąt w silniku cen (ustawowo wolne od pracy zgodnie z przepisami GUS), geokodowanie w oparciu o OpenStreetMap (Nominatim) oraz dane POI z Overpass API z naciskiem na atrakcje turystyczne charakterystyczne dla polskich regionów.

Walutą rozliczeniową jest **złoty polski (PLN)**. Cennik domyślnie uwzględnia szczyty turystyczne specyficzne dla rynku polskiego: wakacje letnie, ferie zimowe, długie weekendy majowe, Boże Narodzenie i Nowy Rok. Administratorzy mogą dodatkowo aktywować nieoczywiste szczyty (np. Andrzejki, Dzień Kobiet).

3. Architektura systemu

3.1 Widok ogólny — warstwy systemu

System jest zorganizowany w cztery logiczne warstwy. Poniższy diagram (uproszczony) pokazuje przepływ danych i odpowiedzialności każdej z nich.

Warstwa	Główna technologia	Odpowiedzialność
Warstwa prezentacji	Next.js 14 (React 18, TypeScript)	SSR i CSR stron, App Router, BFF proxy
Warstwa API	Django REST Framework + Daphne ASGI	Endpointy REST, WebSocket, autoryzacja JWT
Warstwa logiki	Django services + Celery + Channels	Logika biznesowa, zadania asynchroniczne, broadcasting
Warstwa danych	PostgreSQL 16 + PostGIS 3.4 + Redis 7	Persystencja, zapytania geo, cache, broker

Diagram zależności komponentów:

```
                    ↕ HTTPS
Next.js 14 (SSR + CSR) | Route Handler /api/v1/... (BFF proxy)
                    ↕ HTTP REST ↕ WebSocket
                    Daphne ASGI
Django REST Framework | Django Channels (WebSocket)
                    ↕ ORM / PostGIS ↕ Channel layer ↕ Cache/Queue
PostgreSQL + PostGIS | Redis | Celery Worker + Beat
                    ↕ HTTP API
Nominatim (geo) | Overpass (POI) | OpenAI (AI) | Google OAuth | SMTP
```

3.2 Przepływ typowego żądania REST

1	Użytkownik wykonuje akcję w przeglądarce (np. klika Szukaj)
2	Next.js kieruje żądanie do własnego Route Handlera (BFF) pod /api/v1/...
3	BFF przekazuje żądanie do Django (wewnętrzny URL: INTERNAL_API_URL lub DJANGO_API_URL)
4	Daphne przyjmuje żądanie, middleware JWT weryfikuje token z nagłówka Authorization
5	Django sprawdza uprawnienia i throttling, następnie wykonuje logikę w serwisie
6	Serwis wywołuje model/ORM, ewentualnie odpytuje Redis (cache) lub kolejkuje zadanie Celery
7	Odpowiedź JSON wraca przez Daphne → BFF → Next.js → przeglądarka

Dlaczego BFF? *Warstwa Backend-for-Frontend w Next.js rozwiązuje kilka problemów naraz: unika wystawiania adresu wewnętrznego Django bezpośrednio do przeglądarki, upraszcza obsługę błędów sieciowych (np. Django niedostępne → kod UPSTREAM_UNAVAILABLE), oraz pozwala na spójne SSR bez żonglerki CORS-ami w środowisku Dockera.*

3.3 Komunikacja w czasie rzeczywistym (WebSocket)

Czat między gościem a gospodarzem działa przez **WebSockety**. Połączenia są obsługiwane przez Django Channels z dedykowanym consumerem asynchronicznym. Każda konwersacja ma własny kanał (*group*) w Redis — wiadomości są rozsyłane do wszystkich otwartych okien obydwu uczestników.

Parametr	Wartość
Adres połączenia	ws[s]://host/ws/conversations/{uuid}/
Autoryzacja	Token JWT w query string: ?token=
Typy zdarzeń wysyłanych przez klienta	message.send, typing.start, typing.stop, message.read
Typy zdarzeń odbieranych przez klienta	message.new, typing.start, typing.stop, message.read
Warstwa kanałów	Redis 7 (channels-redis)
Serwer	Daphne (ASGI) — obsługuje jednocześnie HTTP i WebSocket

Synchronizacja zakładek przeglądarki *— frontend implementuje AuthCrossTabSync: wylogowanie w jednej zakładce powoduje wylogowanie we wszystkich otwartych kartach tej samej przeglądarki, co chroni przed pozostawieniem aktywnej sesji.*

3.4 Zadania asynchroniczne (Celery)

Celery obsługuje dwie kategorie zadań: **asynchroniczne** (wywoływane natychmiast po zdarzeniu, np. wysłanie e-maila po potwierdzeniu rezerwacji) oraz **cykliczne** (harmonogram Celery Beat). Broker i backend wyników to Redis.

Harmonogram	Zadanie
Co 12 minut	Sprzątanie wygasłych sesji AI (cleanup_expired_ai_sessions)
Co 22 minuty	Usuwanie wygasłych sesji porównywarki (cleanup_expired_compare_sessions)
Co 30 minut	Anulowanie porzuconych rezerwacji AWAITING_PAYMENT → ABANDONED
Co godzinę	Auto-odrzucenie prośb hosta po przekroczeniu okna decyzyjnego → REJECTED
Codziennie 10:00	Przypomnienia e-mail o wystawieniu recenzji (7 dni po wymeldowaniu)
Codziennie 03:05	Odświeżanie cache POI z Overpass API (dane starsze niż 24h)
Codziennie 04:10	Odświeżanie podsumowań tekstowych obszarów (dane starsze niż 7 dni)
1. dnia miesiąca 06:00	Raport kosztów API OpenAI (transparentność operacyjna)
Na żądanie (async)	Wysyłka e-maila po rezerwacji, przeliczenie średniej oceny oferty, ślepe ujawnienie recenzji

Wszystkie zadania cykliczne działają w strefie czasowej **Europe/Warsaw**.

4. Stack technologiczny

4.1 Backend (backend/)

Obszar	Technologia	Uwagi
Język	Python 3.12	Pełna zgodność z obrazem Docker; CI używa 3.12
Framework	Django 5.1.4	ORM, administracja, system uprawnień, signals
REST API	Django REST Framework 3.x	Widoki API, serializery, paginacja, throttling
Dokumentacja API	drf-spectacular	OpenAPI 3.0 + Swagger UI pod /api/schema/swagger-ui/
GIS	GeoDjango + PostGIS 3.4	Zapytania przestrzenne, PointField, GiST indeksy
Autentykacja	djangorestframework-simplejwt	Access token (60 min) + refresh (30 dni), rotacja
OAuth	google-auth	Weryfikacja tokenów Google, auto-tworzenie konta
Realtime	django-channels + channels-redis + daphne	WebSocket czatu, consumer asynchroniczny
Zadania asynchroniczne	Celery 5 + django-celery-beat	Worker + harmonogram Beat
Cache	django-redis + redis-py	Cache zapytań, sesje porównywarki, limity AI
Pliki / S3	boto3 + django-storages	Opcjonalny storage S3; lokalnie MEDIA_ROOT
Walidacja plików	Pillow + python-magic	Sprawdzanie typów MIME, bezpieczne uploady obrazów
Monitoring (prod.)	sentry-sdk	Raportowanie błędów i wyjątków do Sentry
Testy	pytest + pytest-django	Testy jednostkowe i integracyjne z bazą PostGIS
Linting	ruff	Szybki linter i formatter zgodny z PEP 8

4.2 Frontend (frontend/)

Obszar	Technologia	Uwagi
Framework	Next.js 14 (App Router)	SSR, ISR, RSC; React 18; TypeScript
Stylowanie	Tailwind CSS	Utility-first CSS, projekt responsywny
Komponenty UI	Radix UI (dialog, popover)	Dostępne, nieostylowane prymitywy
Mapy	Leaflet + react-leaflet + Leaflet.markercluster	Interaktywna mapa wyszukiwania
Dane geograficzne (statyczne)	GeoJSON (granice PL, województwa, miasta)	Renderowane na mapie jako overlay
Formularze	react-hook-form + zod + @hookform/resolvers	Walidacja po stronie klienta
Stan globalny	Zustand	Lekki store, bez boilerplate Redux
Tokeny JWT (frontend)	jose	Weryfikacja tokenów w Next.js middleware
Daty	date-fns	Formatowanie, obliczenia dat
Animacje	framer-motion	Przejścia, animacje UI
Notyfikacje	react-hot-toast	Toast notifications
Testy E2E	Playwright	Testy end-to-end przeglądarki

4.3 Infrastruktura i narzędzia deweloperskie

Narzędzie	Opis
Docker Compose	Lokalny stack: postgis:16-3.4, redis:7-alpine, backend (Daphne), frontend (Node 20)
Profil Celery	Opcjonalny: docker compose --profile celery up (worker + beat)
Makefile	Skróty: make dev, make migrate, make seed, make test, make lint, make superuser
GitHub Actions	CI: ruff + pytest z usługami PostGIS i Redis na gałęziach main/develop
Seed data	Polecenie manage.py seed_db — ładuje dane demo do lokalnej bazy

5. Moduły domenowe backendu

Backend jest podzielony na aplikacje Django (apps/), każda odpowiadająca jednej domenie biznesowej. Wzorzec implementacji jest konsekwentny: **widok** → **serializer** → **serwis** → **model**, co oznacza, że logika biznesowa nie jest rozrzucona po widokach ani serializerach, lecz enkapsulowana w warstwach serwisowych.

Moduł	Domena	Zakres odpowiedzialności
users	Użytkownicy i tożsamość	Model User z e-mailem jako loginem, UserProfile (bio, avatar, preferencje językowe), WishlistItem (lista życzeń), SavedSearch (zapisane wyszukiwania). Rejestracja, logowanie e-mail/hasło i Google OAuth.
common	Współdzielone komponenty	BaseModel (UUID, soft delete, timestamps), AuditLog (ślad działań dla compliance), wyjątki API ze standardowymi kodami błędów StayMap, paginacja, middleware WebSocket JWT, EmailService, health check endpoint.
listings	Oferty noclegowe	Model Listing (status DRAFT→PENDING→APPROVED/REJECTED/ARCHIVED), ListingLocation (PointField z tagami geograficznymi: near_lake, near_mountains, near_sea, near_forest, beach_access, ski_slopes_nearby, quiet_rural, historic_center_nearby, cycling_routes_nearby), ListingImage, BlockedDate, kalendarz cen/dostępności, destination_score_cache.
pricing	Silnik cenowy	CustomDatePrice (nadpisanie ceny na konkretny dzień), SeasonalPricingRule (zakres dat + mnożnik), HolidayPricingRule (święta), LongStayDiscountRule (rabat za długi pobyt). PricingService oblicza pełny breakdown dla podanych dat i gości.
bookings	Rezerwacje	Model Booking (statusy: PENDING, AWAITING_PAYMENT, CONFIRMED, COMPLETED, REJECTED, CANCELLED, ABANDONED), pricing_breakdown (JSON — snapshot ceny zamrożony w momencie transakcji), BookingStatusHistory, Payment, StripeWebhookEvent (infrastruktura pod płatności).
search	Wyszukiwanie	SearchOrchestrator z geo-filtrowaniem (PostGIS distance), filtrami tagowymi, trybami podróży z scoringiem (TravelModeRanker), 5-minutowy cache Redis. Geokodowanie przez Nominatim.

Moduł	Domena	Zakres odpowiedzialności
host	Panel gospodarza	Onboarding, CRUD ofert, upload zdjęć (z throttlingiem), wysłanie do moderacji, zarządzanie rezerwacjami, reguły cenowe, powiadomienia, statystyki zarobków, recenzje po stronie hosta.
moderation	Moderacja treści	Kolejka ofert w statusie PENDING; akcje: approve (→ APPROVED) lub reject (→ REJECTED + komentarz). Dostępne tylko dla is_admin=True.
reviews	Recenzje	Mechanizm ślepego ujawnienia (oba uczestniki oceniają się niezależnie; recenzje ujawniane jednocześnie), subskory (czystość, lokalizacja, komunikacja, dokładność), odpowiedź hosta, przeliczanie average_rating oferty (Celery).
messaging	Czat	Conversation (1:1 per listing-gość), Message z read_at, MessageTemplate (szablony z podstawianiem zmiennych: {{guest_name}}, {{listing_title}}), WebSocket consumer asynchroniczny.
ai_assistant	Asystent AI	AiTravelSession (TTL 24h), AiTravelPrompt (surowe zapytanie), AiFilterInterpretation (sparsowane parametry), AiRecommendation (oferta + wyjaśnienie dopasowania). Obsługa OpenAI i kompatybilnych API (np. Groq).
discovery	Odkrywanie ofert	DiscoveryCollection (kuratorowane kolekcje na stronę główną, sort_order, tryb podróży), CompareSession (maks. 3 oferty, TTL 48h, jedna sesja na zalogowanego użytkownika).
location_intelligence	Inteligencja lokalizacji	NearbyPlaceCache (wyniki Overpass, TTL 24h), AreaSummaryCache (opis obszaru tekstowy, TTL 7 dni). Scoring atrakcyjności okolicy (0–100 per tryb podróży) przechowywany w Listing.destination_score_cache.

Model bazowy i soft delete

Każdy model dziedziczy po **BaseModel** z apps/common/. Klucz główny to **UUID** — nie auto-inkrementowany integer — co eliminuje ryzyko przewidywalnych ID w API i ułatwia przyszłą migrację między bazami. Pole deleted_at implementuje **soft delete**: rekord jest oznaczany jako usunięty zamiast fizycznego kasowania. Domyślny manager objects automatycznie filtruje rekordy z deleted_at IS NULL, a manager all_objects udostępnia pełny zbiór danych (np. dla administracji i audytu).

AuditLog — dedykowany model w apps/common/ — rejestruje kluczowe zdarzenia systemowe (np. rejestracja przez Google OAuth, logowanie, akcje moderacji) wraz z informacją o użytkowniku, typie akcji i znaczniku czasu. Ślad audytu jest niezbędny z perspektywy compliance i wsparcia technicznego.

6. Funkcjonalności — opis dla zarządu

6.1 Konta i tożsamość użytkowników

Platforma obsługuje dwa sposoby rejestracji i logowania: klasyczne konto e-mail/hasło oraz logowanie kontem Google (OAuth 2.0). Oba mechanizmy zwracają te same tokeny JWT, więc reszta systemu jest niezależna od metody autentykacji.

- **Rejestracja e-mail** — unikalny adres e-mail jest loginem; hasło hashowane przez Django
- **Google OAuth** — token od Google weryfikowany po stronie backendu; jeśli konto z tym e-mailem już istnieje, następuje automatyczne powiązanie
- **JWT** — access token (60 min) przechowywany w ciasteczku HTTP-only; refresh token (30 dni) z rotacją zapewnia przedłużenie sesji bez ponownego logowania
- **Role użytkownika** — ten sam użytkownik może być jednocześnie gościem i gospodarzem (flagi is_host, is_admin)
- **Profil** (UserProfile) — bio, avatar, preferencje językowe, kraj
- **Lista życzeń** (WishlistItem) — gość zapisuje interesujące oferty; lista dostępna w koncie
- **Zapisane wyszukiwania** (SavedSearch) — przechowuje parametry wyszukiwania do ponownego użycia
- **Synchronizacja zakładek** — wylogowanie w jednej karcie przeglądarki wylogowuje wszystkie (AuthCrossTabSync)

6.2 Oferty i wyszukiwanie

Wyszukiwanie to centralna funkcja produktu. Użytkownik może wpisać nazwę miejscowości, wybrać tryb podróży i zobaczyć oferty zarówno na mapie, jak i na liście. Klasteryzacja markerów zapewnia czytelność mapy przy dużej liczbie wyników.

Cykl życia oferty:

Status	Znaczenie
DRAFT	Szkic tworzony przez hosta; niewidoczny publicznie
PENDING	Po kliknięciu Wyslij do moderacji; oczekuje na zatwierdzenie administratora
APPROVED	Zaakceptowana przez moderatora; widoczna w wynikach wyszukiwania
REJECTED	Odrzucona z komentarzem; host może edytować i ponownie wysłać
ARCHIVED	Wycofana z oferty (zachowana w historii, niewidoczna publicznie)

Tryby podróży i scoring:

Każda oferta może być scorowana względem dziesięciu trybów podróży, co umożliwia personalizację wyników wyszukiwania. TravelModeRanker przypisuje punkty na podstawie udogodnień (amenities) i tagów

geograficznych lokalizacji.

Tryb	Charakterystyka
romantic	Pary — cisza, prywatność, kominek, jacuzzi, wino
family	Rodziny z dziećmi — bezpieczna przestrzeń, plac zabaw, kuchnia
pet	Podróże ze zwierzętami — duży ogród, tereny zielone w okolicy
workation	Praca zdalna — szybkie Wi-Fi, biurko, zaciszne otoczenie
slow	Slow travel — cisza, natura, detox od miejskiego zgiełku
outdoor	Aktywny wypoczynek — szlaki, rowery, kajaki
lake	Mazury i jeziora — dostęp do wody, łódki, rejsy
mountains	Góry — widoki, szlaki górskie, kolej linowa
wellness	Wellness — sauna, basen, spa, masaże

Tagi geograficzne lokalizacji (near_lake, near_mountains, near_sea, near_forest, beach_access, ski_slopes_nearby, quiet_rural, historic_center_nearby, cycling_routes_nearby) są ustawiane przez hosta i wpływają na scoring trybów podróży oraz filtrowanie wyników.

6.3 Silnik cen i rezerwacje

Cennik w StayMap Polska jest jednym z bardziej zaawansowanych elementów systemu. Uwzględnia nie tylko prostą cenę za noc, lecz cały szereg zmiennych: sezonowość, polskie święta, niestandardowe szczyty turystyczne, liczbę gości oraz długość pobytu.

Wzór kalkulacji ceny pobytu:

1. Cena za noc (per noc)	cena_bazowa × mnożnik_sezonowy × mnożnik_świąteczny
2. Suma noclegów	Σ(cena_za_noc) dla każdej nocy pobytu
3. Dopłata za dodatkowych gości	`Σ(cena_noc × liczba_dodatkowych_doros■ych × 10%)`
Dopłata za dzieci	`Σ(cena_noc × liczba_dzieci × 5%)`
4. Suma pośrednia	`pkt 2 + pkt 3`
5. Rabat za długi pobyt	suma_pośrednia × procent_rabatu (jeśli liczba_nocy ≥ próg)
6. Suma po rabacie	suma_pośrednia − rabat
7. Opłata za sprzątanie	`Sta■a kwota ustalona przez hosta`
8. Prowizja platformy	`(suma_po_rabacie + sprz■tanie) × % prowizji (domy■lnie 15%)`
RAZEM DO ZAPŁATY	`suma_po_rabacie + sprz■tanie + prowizja_platformy`

Obsługa polskich świąt:

Typ reguły	Opis
Ustawowe dni wolne (GUS)	Automatyczne — zaszyte w kodzie, bez potrzeby konfiguracji
Dodatkowe szczyty turystyczne	Opcjonalne, aktywowane przez admina: Dzień Kobiet (8 mar), Wszystkich Świętych (1 lis), Andrzejki (30 lis), stosowanie_polskich_szczytów = True
Niestandardowe daty hosta	HolidayPricingRule — host definiuje własne specjalne daty z indywidualnym mnożnikiem
Reguły sezonowe hosta	SeasonalPricingRule — zakres dat + mnożnik (domyślne szczyty: lato 1,12×, zima 1,10×)
Nadpisanie konkretnego dnia	CustomDatePrice — stała cena na wybrany dzień, ignoruje mnożniki

Tryby rezerwacji i stany:

Tryb	Opis
INSTANT (natychmiastowa)	Rezerwacja tworzona ze statusem PENDING→AWAITING_PAYMENT; po opłaceniu → CONFIRMED
REQUEST (na prośbę)	Rezerwacja tworzona ze statusem PENDING; host musi zaakceptować w oknie czasowym (domyślnie 24h); brak odpowiedzi → auto-odrzucenie

> ***Snapshot ceny** — w momencie tworzenia rezerwacji pełny breakdown cenowy jest zapisywany jako JSON (pole pricing_breakdown). Zmiana cennika przez hosta po dokonaniu rezerwacji nie wpływa na już zatwierdzone pobyty. Jest to ważne zarówno z perspektywy prawnej (umowa z gościem), jak i rozliczeń finansowych.*

Automatyzacja stanów Celery: porzucone rezerwacje (niepłacone przez BOOKING_PAYMENT_TIMEOUT_H, domyślnie 1h) są automatycznie anulowane. Prośby do hosta bez odpowiedzi w oknie HOST_REQUEST_ACCEPT_HOURS (domyślnie 24h) są auto-odrzucane. To zmniejsza ilość ręcznej pracy i utrzymuje calendar gościa i hosta w spójnym stanie.

6.4 Panel gospodarza

Gospodarz ma do dyspozycji dedykowany panel z pełnym zestawem narzędzi do zarządzania działalnością. Wejście do panelu wymaga uprzedniego przejścia onboardingu — jednorazowego kroku aktywacji profilu hosta.

- **Onboarding** — jednorazowe uruchomienie profilu hosta (POST /host/onboarding/start/); od tej chwili użytkownik ma rolę hosta
- **Zarządzanie ofertami** — tworzenie, edycja, archiwizacja; upload zdjęć z limitem i throttlingiem
- **Wysłanie do moderacji** — jednym kliknięciem zmiana statusu DRAFT → PENDING
- **Reguły cenowe** — sezonowe, świąteczne, long-stay discount; zarządzanie przez API panelu hosta

- **Kalendarz dostępności** — blokowanie dat (BlockedDate), podgląd rezerwacji na kalendarzu
- **Zarządzanie rezerwacjami** — lista, podgląd szczegółów, akceptacja/odrzucenie (tryb REQUEST)
- **Szablony wiadomości** (MessageTemplate) — predefiniowane odpowiedzi z dynamicznymi zmiennymi ({{guest_name}}, {{listing_title}}) przyspieszające komunikację z gościem
- **Recenzje** — podgląd ocen ofert, możliwość jednorazowej odpowiedzi hosta na każdą recenzję
- **Statystyki zarobków** — widok earnings/payouts (endpoint /host/earnings/)

6.5 Moderacja treści

Zanim oferta trafi do wyników wyszukiwania, musi przejść przez ręczną moderację. Mechanizm zapewnia, że goście widzą tylko zweryfikowane, kompletne oferty.

- Administrator widzi kolejkę ofert w statusie PENDING
- Może zatwierdzić ofertę (→ APPROVED, natychmiastowa widoczność w wynikach) lub odrzucić z komentarzem (→ REJECTED)
- Host otrzymuje informację o odrzuceniu wraz z uzasadnieniem i może edytować ofertę i wysłać ją ponownie
- Endpointy moderacji są zabezpieczone uprawnieniem is_admin=True

6.6 System recenzji — mechanizm ślepego ujawnienia

System recenzji zapobiega sytuacji, w której jedna ze stron odczyta ocenę drugiej i dopiero wtedy napisze swoją własną. Rozwiązanie opiera się na **mechanizmie ślepego ujawnienia** (blind release):

Krok 1: Gość ocenia pobyt (recenzja GUEST, is_public=False)

Krok 2: Host ocenia gościa (recenzja HOST, is_public=False)

Krok 3: Gdy obydwie strony złożyły recenzje, Celery task release_blind_review_task ujawnia je jednocześnie (is_public=True) w atomowej transakcji

Krok 4: Jeśli jedna ze stron nie oceni w oknie czasowym (blind_release_at), jej recenzja pozostaje ukryta; recenzja istniejącej strony jest ujawniana po terminie

Recenzja gościa zawiera ogólną ocenę (1–5 gwiazdek) i opcjonalne subskory: **czystość, lokalizacja, komunikacja z gospodarzem, zgodność z opisem**. Subskory są agregowane do średniej w modelu oferty (average_subscores). Host może **raz** odpowiedzieć na recenzję gościa — odpowiedź jest widoczna publicznie.

> ***Przypomnienia e-mail*** *— 7 dni po wymeldowaniu Celery rozsyła przypomnienia do gości, którzy nie wystawili jeszcze oceny. Zwiększa to liczbę recenzji bez irytowania gościa zbyt wczesnymi powiadomieniami.*

6.7 Komunikacja i czat w czasie rzeczywistym

Każda para gość–oferta może mieć jedną konwersację. Wiadomości są dostępne zarówno przez REST API (historia, tworzenie), jak i przez WebSocket (powiadomienia na żywo).

- Konwersacja tworzona jest automatycznie przy pierwszym kontakcie gościa z daną ofertą

- REST API: lista konwersacji z licznikiem nieprzeczytanych, historia wiadomości
- WebSocket: nowe wiadomości pojawiają się natychmiastowo bez odświeżania strony
- Wskaźniki pisania (typing indicators) i potwierdzenia przeczytania (read receipts)
- Szablony wiadomości dla hosta — gospodyni może szybko odpowiedzieć gotowym szablonem z podstawionymi danymi gościa i oferty
- Licznik nieprzeczytanych wiadomości widoczny w nawigacji aplikacji

6.8 Asystent AI — wyszukiwanie w języku naturalnym

Moduł AI pozwala użytkownikowi wpisać zapytanie po polsku, tak jak zadałby pytanie człowiekowi — bez wypełniania formularzy z filtrami. System sam interpretuje intencję i przekłada ją na konkretne parametry wyszukiwania.

Etap	Opis
Zapytanie uzytkownika	Szukam spokojnego domku dla 4 osob na Mazurach na tydzien w sierpniu, z sauna, do 300 zl/noc
Parsowanie przez LLM	Model jezykowy (OpenAI gpt-4o-mini lub kompatybilny) wyodrebnia: lokalizacja=Mazury, tryb=lake, goscie=4, daty=sierpien, amenity=sauna, max_price=300
Wyszukiwanie	SearchOrchestrator uruchamia geo-filtrowanie + scoring trybow podrozy bez cache (zawsze swieze dane)
Wyjasnienie dopasowania	Drugi call do LLM generuje 2-3 zdaniowe wyjasnienie po polsku dla kazdej rekomendacji + lista wyroznikow
Wynik	Frontend wyswietla oferty z kartami wyjasnienia: Ten domek spelnia Twoje oczekiwania, poniewaz...

Sesje AI mają TTL 24h (konfigurowalny). Użytkownik może zadawać pytania uzupełniające w tej samej sesji (follow-up prompts). Stare sesje są usuwane automatycznie przez zadanie Celery (co 12 minut). Koszty wywołań API są monitorowane w modelu sesji i raportowane cyklicznie (miesięczny raport kosztów).

> *System jest przygotowany do pracy zarówno z OpenAI jak i z alternatywami zgodnymi z protokołem OpenAI API (np. **Groq** z modelami Llama — znacznie tańsza alternatywa). Zmiana dostawcy wymaga jedynie ustawienia OPENAI_BASE_URL i OPENAI_MODEL.*

6.9 Odkrywanie ofert i porównywarka

Moduł discovery obsługuje dwa niezależne scenariusze:

- **Kolekcje kuratorowane** (DiscoveryCollection) — administrator tworzy tematyczne zestawy ofert (np. Gorskie chaty z sauna, Last minute Baltyk), które pojawiają się na stronie głównej z określoną kolejnością i ewentualnym filtrem trybu podróży
- **Oferty last-minute** — feed dynamicznych propozycji z krótkim terminem (integracja z discovery feed API)

Porównywarka (Compare):

- Gość może zebrać do 3 ofert w sesji porównawczej
- Sesja działa dla użytkowników anonimowych (klucz sesji w localStorage) i zalogowanych (1 aktywna sesja per użytkownik)
- Zestawienie ofert w tabeli side-by-side: cena, lokalizacja, amenity, oceny
- Sesja wygasa po 48h (konfigurowalny TTL); wygasłe sesje czyszczone przez Celery co 22 minuty
- Maksymalnie 3 oferty egzekwowane zarówno po stronie API jak i frontu

6.10 Inteligencja lokalizacji

Każda oferta może być wzbogacona o automatycznie generowane informacje o okolicy — bez manualnego opisywania przez hosta. Dane pochodzą z OpenStreetMap (Overpass API) i są grupowane w kategorie tematyczne.

Kategoria POI	Przykłady
eat_drink	Restauracje, kawiarnie, puby
nature_leisure	Parki, rezerwaty, tereny rekreacyjne
family	Atrakcje dla dzieci, place zabaw, aquaparki
culture	Muzea, zabytki, galerie
transport	Stacje kolejowe, przystanki, lotniska
outdoor	Szlaki turystyczne, wypożyczalnie rowerów, kajakarnie
services	Sklepy, apteki, bankomaty

Wyniki Overpass są cache'owane przez 24h (NearbyPlaceCache) i odświeżane nocą przez Celery. Na podstawie gęstości POI w poszczególnych kategoriach obliczany jest **Destination Score** (0–100 punktów per tryb podróży), który wpływa na ranking ofert w wyszukiwarce. Tekstowe podsumowanie okolicy (AreaSummaryCache, TTL 7 dni) jest wyświetlane na karcie oferty.

7. Frontend — mapa aplikacji

Frontend zbudowany jest w Next.js 14 z App Router, co oznacza, że każda trasa może być renderowana po stronie serwera (SSR), po stronie klienta (CSR) lub jako statyczna strona (SSG) — zależnie od potrzeb. Aplikacja podzielona jest na trzy grupy tras (route groups), co odwzorowuje trzy konteksty użytkowania.

Grupa tras	Przykładowe URL	Charakter
(auth)	/login, /register	Logowanie, rejestracja; bez paska nawigacji hosta
(main)	/, /search, /listing/[slug], /bookings, /account, /messages, /compare, /ai, /discovery, /wishlist, /travels	Główna witryna dla gościa; SSR dla SEO na stronach ofert
(host)	/host/panel, /host/listings, /host/listings/new, /host/bookings, /host/messages, /host/reviews, /host/pricing, /host/earnings, /host/settings	Panel gospodarza; wymaga is_host=True; dostęp chroniony middleware

Kluczowe strony i ich funkcje

Strona / funkcja	Zawartość
Strona główna (/)	Hero z wyszukiwarką; kafle regionów (Góry, Bałtyk, Jeziora, Uzdrowiska); kafle trybów podróży; teaser AI; kuratorowane kolekcje; ostatnia szansa (3D carousel last-minute)
Wyszukiwanie (/search)	Mapa Leaflet z klasteryzacją markerów; panel filtrów (lokalizacja, daty, goście, tryb, cena, udogodnienia); paginacja cursorowa; ładowanie większej liczby wyników przy scrollu
Karta oferty (/listing/[slug])	Galeria zdjęć; opis, udogodnienia, polityka anulowania; widget rezerwacyjny z wyceną live; recenzje z podskores; opis okolicy + POI; kalendarz cen
Przepływ rezerwacji	Widget na karcie oferty → wycena (POST /bookings/quote/) → potwierdzenie → sukces/status
Konto gościa (/account)	Dane osobowe, avatar, preferencje, historia rezerwacji, lista życzeń, zapisane wyszukiwania
Wiadomości (/messages)	Lista konwersacji (licznik nieprzeczytanych); WebSocket dla wiadomości na żywo; wskaźniki pisania i potwierdzenia przeczytania

Strona / funkcja	Zawartość
Porównywarka (/compare)	Do 3 ofert side-by-side; możliwość dodania z karty oferty lub wyszukiwarki; sesja persystentna (anonimowa i zalogowana)
Asystent AI (/ai)	Pole tekstowe; przetwarzanie zapytania; wyniki z wyjaśnieniami dopasowania; historia sesji (/ai/history)
Discovery (/discovery)	Kolekcje tematyczne; oferty last-minute; filtry trybu podróży
Panel hosta (/host/panel)	Dashboard z powiadomieniami; quick-stats; linki do podsekcji
Oferty hosta (/host/listings)	Lista ofert z statusami; CRUD; upload zdjęć; wysyłanie do moderacji
Rezerwacje hosta (/host/bookings)	Lista z filtrem statusu; akcja akceptuj/odrzuć dla trybu REQUEST; podgląd szczegółów
Cennik hosta (/host/pricing)	Tworzenie/edycja reguł sezonowych, świątecznych i long-stay
Moderacja (admin)	Widoczna w panelu hosta dla is_admin=True; kolejka PENDING; akcje zatwierdź/odrzuć z komentarzem

BFF (Backend for Frontend)

Plik frontend/src/app/api/v1/[...path]/route.ts pełni rolę cienkiej warstwy proxy między przeglądarką a Django. Wszystkie żądania REST z frontu przechodzą przez ten handler, który:

- Przekazuje nagłówek Authorization (JWT) do Django
- Obsługuje GET, POST, PUT, PATCH, DELETE, HEAD
- Prawidłowo przesyła zarówno JSON jak i multipart/form-data (upload plików)
- Zwraca HTTP 503 z kodem UPSTREAM_UNAVAILABLE jeśli Django jest nieosiągalne (pomocne w dev)
- Zachowuje nagłówki Rate-Limit i Retry-After z odpowiedzi Django

Zmienne środowiskowe frontu: *NEXT_PUBLIC_API_URL (adres dla przeglądarki, np. https://api.staymappolska.pl) i INTERNAL_API_URL (adres wewnętrzny Dockera dla SSR, np. http://backend:8000). JWT_SECRET musi być identyczny z Django SECRET_KEY — inaczej middleware Next.js nie zweryfikuje tokenu i użytkownik zostanie wylogowany przy każdym renderowaniu serwerowym.*

Statyczne dane geograficzne

W katalogu frontend/public/data/maps/ przechowywane są pliki GeoJSON: granica Polski, podziały na województwa oraz punkty głównych miast. Są one ładowane przez Leaflet jako dodatkowe warstwy overlay na mapie wyszukiwania — użytkownik widzi wyraźne granice regionów i może łatwiej zlokalizować, gdzie szuka noclegu.

8. API — przegląd endpointów

Pełna dokumentacja API jest dostępna jako Swagger UI pod adresem /api/schema/swagger-ui/ (generowana automatycznie przez drf-spectacular). Poniżej zestawienie kluczowych grup endpointów z krótkim opisem — dla orientacji biznesowej i technicznej.

Grupa	Kluczowe endpointy	Co realizuje
Autentykacja	POST /auth/register/ POST /auth/token/ POST /auth/token/refresh/ POST /auth/google/ GET/PATCH /auth/me/	Rejestracja, logowanie (email+hasło i Google OAuth), odśwież token JWT, profil zalogowanego użytkownika
Oferty (publiczne)	GET /listings/search/ GET /listings/{slug}/ GET /listings/{slug}/price-calendar/	Wyszukiwanie z geo-filtrowaniem, szczegóły oferty (slug dla SEO), podgląd kalendarza cen
Rezerwacje (gość)	POST /bookings/quote/ POST /bookings/ GET /bookings/me/ GET /bookings/{uuid}/ DELETE /bookings/{uuid}/	Wycena (bez tworzenia rezerwacji), nowa rezerwacja, lista swoich rezerwacji, szczegóły, anulowanie
Recenzje	POST /reviews/ PATCH /reviews/{uuid}/host-response/	Wystawienie recenzji (gość lub host), odpowiedź hosta na recenzję gościa
Wiadomości	GET/POST /conversations/ GET/POST /conversations/{uuid}/messages/ WS /ws/conversations/{uuid}/	Lista/tworzenie konwersacji, historia/wysyłka wiadomości, WebSocket dla komunikacji na żywo
Panel hosta	POST /host/onboarding/start/ GET/POST /host/listings/ POST /host/listings/{uuid}/images/ POST /host/listings/{uuid}/submit-for-review/ GET /host/bookings/ PATCH /host/bookings/{uuid}/status/	Pełny CRUD ofert, upload zdj., wysyłanie do moderacji, zarządzanie rezerwacjami, reguły cenowe, statystyki
Moderacja (admin)	GET /admin/moderation/listings/ POST .../approve/ POST .../reject/	Kolejka PENDING, akceptacja, odrzucenie z komentarzem — tylko is_admin=True

Grupa	Kluczowe endpointy	Co realizuje
Discovery i Compare	GET /discovery/ GET/POST /compare/ POST /compare/bootstrap/ POST /compare/listings/ DELETE /compare/listings/{uuid}/	Feed strony głównej, sesja porównawcza, dodawanie/usuwanie ofert do porównania
AI i wyszukiwanie	GET /search/ POST /ai/search/ GET /ai/search/{session_id}/ POST /ai/search/{session_id}/prompt/	Zaawansowane wyszukiwanie, nowa sesja AI, podgląd sesji, pytanie uzupełniające w sesji
Konto użytkownika	POST /wishlist/ GET /saved-searches/ POST /saved-searches/	Dodawanie/usuwanie z listy życzeń, zarządzanie zapisanymi wyszukiwaniami
System	GET /health/ GET /api/schema/ GET /api/schema/swagger-ui/	Health check, schemat OpenAPI, Swagger UI

Standardy API

Wszystkie odpowiedzi używają ujednoliconego formatu JSON. Błędy mają standaryzowane kody (np. UPSTREAM_UNAVAILABLE, BOOKING_CONFLICT, LISTING_NOT_APPROVED), co ułatwia obsługę po stronie frontu i diagnostykę. Throttling jest skonfigurowany na wrażliwych endpointach (upload, operacje AI, endpointy autoryzacji). Paginacja jest cursorowa (wydajniejsza od offset dla dużych zbiorów).

9. Bezpieczeństwo

Poniżej zestawienie warstw bezpieczeństwa wbudowanych w architekturę systemu. Nie są to funkcje dodane po fakcie — większość wynika z priorytetów projektowych.

Obszar	Mechanizm	Wartość bezpieczeństwowa
Autentykacja	JWT (access 60 min, refresh 30 dni) w cookie HTTP-only; rotacja refresh tokenów; brak sesji serwerowej	Tokeny nie są dostępne przez JavaScript (XSS mitigation); rotacja minimalizuje ryzyko przejęcia refresh tokenu
Autoryzacja	Uprawnienia per endpoint: IsAuthenticated, IsHost, IsAdmin, IsAuthenticatedOrReadOnly; izolacja danych użytkownika	Gość nie może edytować cudzych ofert; host nie widzi rezerwacji innych hostów
Soft delete	Rekordy oznaczane jako usunięte (deleted_at) zamiast fizycznego kasowania	Ochrona przed przypadkową utratą danych; możliwość przywrócenia przez admina
Audit Log	AuditLog zapisuje kluczowe zdarzenia (rejestracja Google, logowanie, akcje moderacji) z user_id, action_type, timestamp	Ślad audytu dla compliance i wsparcia; diagnostyka incydentów bezpieczeństwa
Walidacja uploadów	Pillow weryfikuje zawartość plików graficznych; python-magic sprawdza rzeczywisty typ MIME (nie tylko rozszerzenie)	Ochrona przed przesyłaniem szkodliwych plików ukrytych pod *.jpg
Throttling	Limity na endpointach autoryzacji, uploadu zdj. i AI — konfigurowane przez DRF throttle classes	Ochrona przed brute-force i kosztownymi nadużyciami AI
Sentry (prod)	Automatyczne przechwytywanie wyjątków i błędów 500 z pełnymi stack trace	Szybka diagnostyka problemów produkcyjnych bez dostępu do logów serwera

Obszar	Mechanizm	Wartość bezpieczeństwowa
Synchronizacja zakładek	AuthCrossTabSync wylogowuje wszystkie karty przeglądarki przy wylogowaniu w jednej	Zapobiega pozostawieniu aktywnej sesji na współdzielonym komputerze
UUID jako klucz główny	Wszystkie ID w API są UUID zamiast auto-inkrementowanych integerów	Eliminuje przewidywalne ID (IDOR mitigation); ułatwia migracje danych

Uwaga konfiguracyjna: *JWT_SECRET po stronie Next.js musi być identyczny z Django SECRET_KEY. Niespójność tych wartości powoduje, że middleware Next.js nie może zweryfikować tokenu z ciasteczka — efektem są błędy 500 przy SSR. Należy to sprawdzić przed wdrożeniem produkcyjnym.*

10. Operacje i wdrożenie produkcyjne

10.1 Komponenty wymagane na produkcji

Komponent	Uwagi produkcyjne
PostgreSQL 16 + PostGIS	Źródło prawdy; wymaga rozszerzenia postgis i btree_gist; zalecane managed DB z automatycznym backupem
Redis 7	Cache zapytań, broker Celery, warstwa kanałów WebSocket; jedna instancja obsługuje wszystkie trzy role
Celery Worker	Przynajmniej jeden worker obsługujący kolejkę domyślną; w produkcji można skalować horyzontalnie
Celery Beat	Jeden scheduler (singleton); uruchamianie więcej niż jednego beat spowodowałoby duplikaty zadań
Daphne / Gunicorn	Dev: Daphne obsługuje HTTP+WS. Prod: Gunicorn dla HTTP i osobny Daphne/Uvicorn dla WS, lub unified ASGI
Nginx (reverse proxy)	TLS termination, serwowanie plików statycznych (WhiteNoise jako fallback), proxying do Gunicorn i Daphne
SMTP	Transakcyjne e-maile: potwierdzenia rezerwacji, przypomnienia recenzji; zalecany Gmail App Password lub SendGrid
S3 (opcjonalnie)	Skalowalny storage dla zdjęć ofert; przy USE_S3=False pliki lądują lokalnie w MEDIA_ROOT
Sentry	DSN konfigurowany przez SENTRY_DSN i SENTRY_ENVIRONMENT; tylko w produkcji (production.txt)

10.2 Kluczowe zmienne środowiskowe

Pełna lista z komentarzami jest w pliku .env.example w repozytorium.

Zmienna	Opis
SECRET_KEY	Silny losowy klucz Django; również używany jako JWT_SECRET po stronie Next.js
DATABASE_URL	postgres://user:pass@host:5432/dbname — musi być z PostGIS
REDIS_URL	redis://:pass@host:6379/0

Zmienna	Opis
OPENAI_API_KEY	Klucz OpenAI lub kompatybilnego dostawcy (np. Groq)
OPENAI_BASE_URL	Puste dla OpenAI; https://api.groq.com/openai/v1 dla Groq
OPENAI_MODEL	Domyślnie gpt-4o-mini (tańszy); można ustawić gpt-4o lub modele Groq
GOOGLE_OAUTH_CLIENT_ID	Z Google Cloud Console; wymagany dla logowania Google
USE_S3 / AWS_*	USE_S3=True aktywuje S3; wymagane: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_STORAGE_BUCKET_NAME
EMAIL_HOST / EMAIL_HOST_PASSWORD	SMTP host i hasło (dla Gmail: 16-znakowe App Password z 2FA)
PLATFORM_SERVICE_FEE_PERCENT	Prowizja platformy (domyślnie 15); zmiana wymaga przemyślenia modelu biznesowego
HOST_REQUEST_ACCEPT_HOURS	Okno decyzyjne hosta w trybie REQUEST (domyślnie 24h)
AI_SESSION_TTL_HOURS	Czas życia sesji AI (domyślnie 24h)
COMPARE_SESSION_TTL_HOURS	Czas życia sesji porównywarki (domyślnie 48h)

10.3 Proces wdrożenia (opis kroków)

1	Sklonuj repozytorium, skopiuj .env.example do .env i uzupełnij wartości produkcyjne
2	Zbuduj obrazy Docker: docker compose -f docker-compose.prod.yml build
3	Wykonaj migracje bazy: docker compose run --rm backend python manage.py migrate
4	Zbierz pliki statyczne: python manage.py collectstatic --noinput
5	Uruchom stos: docker compose up -d (z profilem celery dla workerów)
6	Uruchom Next.js: npm run build && npm start (lub przez PM2/Docker)
7	Skonfiguruj Nginx jako reverse proxy z TLS (Let's Encrypt / certbot)
8	Zweryfikuj health check: GET /health/ powinien zwrócić HTTP 200
9	Sprawdź Sentry — po pierwszym uruchomieniu wyślij testowy wyjątek
10	Stwórz konto admina: python manage.py superuser lub przez panel Django Admin

10.4 Skalowanie i dostępność

Przy większym ruchu można skalować następujące komponenty niezależnie:

- **Django backend** — wiele instancji Gunicorn za load balancerem (Nginx / AWS ALB); baza danych jest wspólna
- **Celery workers** — wiele workerów pobierających z tej samej kolejki Redis; Beat musi pozostać singletonem
- **Django Channels / WebSocket** — skalowanie horyzontalnie z jednym Redis jako channel layer
- **Redis** — Redis Cluster lub Redis Sentinel dla wysokiej dostępności i failover
- **PostgreSQL** — repliki read-only dla odciążenia zapytań SELECT (wyszukiwanie)
- **Next.js** — możliwość wdrożenia na Vercel, AWS, lub własnej infrastrukturze Node.js

11. Testy i CI/CD

11.1 Backend — testy jednostkowe i integracyjne

Backend testowany jest za pomocą **pytest** z pluginem pytest-django. Testy mogą korzystać z rzeczywistej bazy PostGIS (uruchamianej w Docker na CI), co eliminuje ryzyko ukrycia błędów za mockami.

- Testy jednostkowe serwisów — PricingService, BookingService, AISearchService
- Testy integracyjne endpointów — używają TestClient Django REST Framework
- Testy modeli — właściwości, relacje, soft delete
- Testy zadań Celery — weryfikacja logiki bez rzeczywistego kolejkowania

11.2 Frontend — testy E2E (Playwright)

Playwright uruchamia testy w przeglądarce Chromium/Firefox/WebKit, testując krytyczne ścieżki użytkownika od początku do końca:

- Logowanie i rejestracja (email + Google OAuth)
- Wyszukiwanie ofert i filtrowanie
- Przepływ rezerwacji — od wyboru dat do potwierdzenia
- Tworzenie oferty przez hosta
- Wysyłanie wiadomości w czacie

11.3 Pipeline CI/CD (GitHub Actions)

Aspekt CI	Szczegóły
Wyzwalacz	Push lub pull request do gałęzi main / develop
Usługi kontenerowe w CI	postgis:16-3.4 + redis:7 (uruchamiane automatycznie przez Actions)
Job Backend	1. Lint: ruff check apps/ config/ 2. Pytest: pytest -q --cov z bazą PostGIS Python 3.12, GDAL, libmagic
Job Frontend	npm ci + npx playwright install + playwright test
Brama jakości	Każdy pull request musi przejść linting i testy; merge zablokowany przy błędach

Dzięki temu podejściu każda zmiana w kodzie jest automatycznie weryfikowana. Regresje są wykrywane przed mergem, a nie po wdrożeniu na produkcję. Pipeline działa w około 3–5 minut dla typowego PR.

12. Zależności zewnętrzne

System integruje się z czterema zewnętrznymi usługami. Każda integracja powinna mieć skonfigurowane limity, właściwy User-Agent i monitoring kosztów.

Usługa	Rola w systemie	Uwagi operacyjne
OpenStreetMap / Nominatim	Geokodowanie (adres lub nazwa miejsca → współrzędne GPS) przy wyszukiwaniu; wymagany poprawny User-Agent zgodny z polityką OSM (Nominatim Usage Policy); bezpłatny, ale z limitami częstotliwości	Brak klucza API; ograniczenie: 1 zapytanie/sekundę; nie nadaje się do masowego geokodowania
Overpass API (OSM)	Pobieranie POI (punkty zainteresowania) w zadanym promieniu od oferty; wyniki cache'owane 24h w NearbyPlaceCache; odświeżane nocą przez Celery	Bezpłatny, ale wolny (latencja 1–5s); cache jest kluczowy dla responsywności aplikacji
OpenAI API (lub kompatybilny)	Parsowanie zapytań w języku naturalnym (AI search); generowanie wyjaśnień dopasowania ofert; domyślny model gpt-4o-mini; możliwość zamiany na Groq (Llama) dla niższych kosztów	Koszt zależy od liczby tokenów; monitorowanie przez AiTravelSession.tokens_used + miesięczny raport Celery
Google OAuth 2.0	Logowanie kontem Google; token weryfikowany po stronie backendu przez google-auth; auto-tworzenie konta przy pierwszym logowaniu	Wymaga GOOGLE_OAUTH_CLIENT_ID z Google Cloud Console; należy skonfigurować OAuth consent screen

Resilience: *Zarówno Nominatim jak i Overpass nie gwarantują SLA. Błąd geokodowania nie blokuje wyszukiwania — zapytanie jest obsłużone bez filtru geograficznego z komunikatem dla użytkownika. Cache POI zapewnia, że nawet przy czasowej niedostępności Overpass, dane z ostatnich 24h są serwowane z bazy lokalnej.*

13. Diagramy przepływów

Poniżej uproszczone diagramy najważniejszych przepływów w systemie. Pokazują kolejność kroków, uczestników i kluczowe decyzje.

13.1 Przepływ rezerwacji (tryb INSTANT)

Gość	Wybiera daty, liczbę gości, klika Zarezerwuj
Frontend	POST /bookings/quote/ → wyświetla rozbicie ceny
Gość	Potwierdza rezerwację
Frontend	POST /bookings/ → tworzy rezerwację
Backend	AvailabilityService sprawdza konflikty; jeśli OK → PricingService zapisuje snapshot ceny
Backend	Rezerwacja status: PENDING → AWAITING_PAYMENT
Backend	Celery: wyślij e-mail potwierdzenia gościa + hosta
Gość	Opłaca rezerwację (Stripe lub inny provider)
Backend	Po płatności status: AWAITING_PAYMENT → CONFIRMED
Backend	Celery: auto-anulowanie jeśli płatność nie nastąpi w BOOKING_PAYMENT_TIMEOUT_H
Gość/Host	Pobyt, wymeldowanie → status → COMPLETED
Celery	7 dni po wymeldowaniu: przypomnienie o recenzji

13.2 Przepływ rezerwacji (tryb REQUEST)

Gość	Tworzy prośbę o rezerwację
Backend	Status: PENDING; ustawia host_response_deadline (teraz + HOST_REQUEST_ACCEPT_HOURS)
Host	Widzi prośbę w panelu; ma okno czasowe na decyzję
Host akceptuje	Status: PENDING → AWAITING_PAYMENT; gość opłaca
Host odrzuca	Status: PENDING → REJECTED; Celery powiadamia gościa
Brak odpowiedzi	Celery Beat (co godzinę): auto-odrzucenie po deadline → REJECTED
Po akceptacji	Identycznie jak INSTANT od kroku płatności

13.3 Przepływ moderacji oferty

Host	Tworzy ofertę (DRAFT); uzupełnia informacje, dodaje zdjęcia
Host	Klika Wyślij do moderacji → status: DRAFT → PENDING
Admin	Widzi ofertę w kolejce moderacji; sprawdza zawartość
Admin akceptuje	Status: PENDING → APPROVED; oferta pojawia się w wynikach wyszukiwania
Admin odrzuca	Status: PENDING → REJECTED + komentarz; host może poprawić i wysłać ponownie
Host (po odrz.)	Edytuje ofertę; ponownie klika Wyślij do moderacji

13.4 Przepływ ślepego ujawnienia recenzji

Gość	Po pobycie wystawia recenzję (is_public=False)
Host	Niezależnie wystawia recenzję gościa (is_public=False)
Celery	release_blind_review_task — obie recenzje istnieją? → atomowe ujawnienie obu (is_public=True)
Celery	Jeśli minie blind_release_at i jedna strona nie oceniła — ujawnienie istniejącej recenzji
Host	Może jednorazowo odpowiedzieć na recenzję gościa (PATCH /reviews/{uuid}/host-response/)

14. Otwarte tematy i dalszy rozwój

Projekt jest funkcjonalnie kompletny — wszystkie kluczowe funkcjonalności są zrealizowane i przetestowane. Poniżej uczciwy obraz obszarów, które wymagają decyzji biznesowej lub konfiguracji środowiskowej przed pełnym uruchomieniem produkcyjnym.

Temat	Priorytet	Opis i zalecenie
Płatności online (Stripe)	WYSOKA	Infrastruktura (modele, StripeWebhookEvent, idempotency keys) jest gotowa. Wymaga konfiguracji konta Stripe, webhook URL i STRIPE_SECRET_KEY. Decyzja biznesowa dotycząca modelu prowizji (capture po pobycie vs. przy rezerwacji) musi poprzedzać aktywację.
Konfiguracja e-mail produkcyjnego	WYSOKA	Bez SMTP e-maile nie są wysyłane (potwierdzenia, przypomnienia). Zalecane: Gmail App Password lub SendGrid. Szczegóły w docs/EMAIL_AND_OPS.md.
Konfiguracja Sentry DSN	ŚREDNIA	Bez DSN błędy produkcyjne nie są monitorowane. Konfiguracja zajmuje 15 minut — warto ustawić przed wdrożeniem.
Izochrony (OpenRouteService)	NISKA	Wzmiankowane w starszej specyfikacji; nie ma implementacji w kodzie. Jeśli potrzebne (np. oferty do 2h drogi od Warszawy), wymaga osobnego modułu.
Powiadomienia push (mobile)	NISKA	Czat działa przez WebSocket; nie ma powiadomień push dla użytkownika w tle. Można dodać FCM/APNs jako nowy moduł.
Moduł wypłat dla hosta	ŚREDNIA	Endpoint /host/earnings/ istnieje; logika wypłat (Stripe Connect lub przelew tradycyjny) jest do zaprojektowania.
Testy obciążeniowe	ŚREDNIA	Zalecane przed wdrożeniem produkcyjnym — szczególnie dla wyszukiwania geo i WebSocket (dużo połączeń równoległych).
Analityka użytkowania	NISKA	Brak wbudowanej analityki; można podpiąć Google Analytics, Plausible lub Mixpanel na poziomie Next.js bez zmian w backendzie.

Rekomendacja przed uruchomieniem produkcyjnym: *Skonfigurować SMTP i Sentry (2h pracy), następnie przeprowadzić testy obciążeniowe wyszukiwania (1 dzień), a płatności Stripe uruchomić w fazie beta z ograniczoną liczbą użytkowników.*

15. Słowniczek pojęć

Poniżej definicje terminów technicznych używanych w tym dokumencie, skierowane do osób niekoniecznie znających szczegóły implementacji.

Termin	Definicja
ASGI	Asynchronous Server Gateway Interface — standard serwera Python obsługujący jednocześnie HTTP i WebSocket; wdrożony przez Daphne
BFF	Backend-for-Frontend — cienka warstwa w Next.js proxy'ująca żądania do Django; upraszcza SSR i konfigurację CORS
Celery	Biblioteka Python do asynchronicznych i cyklicznych zadań (np. wysyłka e-maili, czyszczenie danych); broker: Redis
Celery Beat	Scheduler Celery do zadań cyklicznych (odpowiednik crona); musi być singletonem
CI/CD	Continuous Integration / Continuous Deployment — automatyczne testy i wdrożenia po każdej zmianie kodu
Django Channels	Rozszerzenie Django dodające obsługę WebSocket; używane do czatu w czasie rzeczywistym
GeoJSON	Format danych geograficznych oparty na JSON; używany do granic Polski i województw na mapie frontu
GeoDjango / PostGIS	Rozszerzenia Django i PostgreSQL do przechowywania i indeksowania danych geograficznych; umożliwiają zapytania o odległość
JWT	JSON Web Token — token dostępu bez sesji serwerowej; access (60 min) + refresh (30 dni)
LLM	Large Language Model — duży model językowy (np. GPT-4o-mini); używany w module AI do interpretacji zapytań
Nominatim	Silnik geokodowania OpenStreetMap; zamienia nazwę miejsca na współrzędne GPS
OpenAI-compatible API	Interfejs API kompatybilny z OpenAI; pozwala użyć alternatywnych dostawców (np. Groq z Llama) bez zmiany kodu
Overpass API	API do odpytywania danych OpenStreetMap o punkty zainteresowania (POI) w zadanym obszarze
POI	Point of Interest — punkt zainteresowania (restauracja, muzeum, szlak itp.) używany do oceny atrakcyjności okolicy

Termin	Definicja
Redis	Baza klucz-wartość w pamięci; w projekcie: cache zapytań, broker Celery, warstwa kanałów WebSocket
REST API	Architektura API oparta na HTTP; endpointy zwracają JSON; używane przez frontend do komunikacji z backendem
Sentry	Platforma monitorowania błędów; przechwytuje wyjątki produkcyjne z pełnym kontekstem
Soft delete	Oznaczenie rekordu jako usunięty (deleted_at) zamiast fizycznego kasowania; umożliwia przywrócenie danych
SSR	Server-Side Rendering — renderowanie stron HTML na serwerze Next.js; lepsze SEO i czas pierwszego ładowania
Stripe	Płatność online; infrastruktura (modele, klucze) przygotowana w projekcie, oczekuje konfiguracji
Throttling	Ograniczenie liczby zapytań do API w jednostce czasu; ochrona przed brute-force i nadużyciami
TTL	Time to Live — czas życia rekordu w cache lub sesji; po upływie TTL dane są uznawane za przestarzałe
UUID	Universally Unique Identifier — globalnie unikalny identyfikator; klucz główny wszystkich modeli w projekcie
WebSocket	Protokół komunikacji dwukierunkowej w czasie rzeczywistym; używany w czacie między gościem a gospodarzem
WhiteNoise	Biblioteka Python do serwowania plików statycznych przez Django/Gunicorn bez dodatkowego serwera HTTP

Dokumenty powiązane w repozytorium

Plik	Zawartość
README.md	Szybki start (make dev, make migrate, make seed), porty, skrót poleceń
docs/StayMap_Dokumentacja_Profesjonalna.md	Bogata specyfikacja historyczna / code review; nie zastępuje audytu aktualnego kodu
docs/EMAIL_AND_OPS.md	Konfiguracja e-mail, Sentry, Celery, testy E2E, migracje
.env.example	Pełna lista zmiennych środowiskowych z komentarzami

Plik	Zawartość
AI_SEARCH_TEST.md	Podręcznik manualnego testowania modułu AI
OPENAI_KEY_FIX.md	Przewodnik rozwiązywania problemów z integracją OpenAI
.github/workflows/ci.yml	Definicja pipeline CI/CD (GitHub Actions)

StayMap Polska to kompletna platforma rezerwacji noclegów z mapą, silnikiem cen uwzględniającym polski kalendarz, panelami gościa i gospodarza, moderacją, czatem na żywo i modułami AI oraz discovery — zbudowana na sprawdzonym stosie Django 5 / PostGIS + Next.js 14, gotowa do utrzymania i dalszego rozwoju pod nadzorem operacyjnym (CI/CD, Sentry, harmonogram zadań Celery).

Dokument sporządzony na podstawie analizy struktury repozytorium, konfiguracji środowiskowej i kodu źródłowego (stan: kwiecień 2026). Wszelkie decyzje biznesowe dotyczące wdrożenia produkcyjnego powinny być poprzedzone weryfikacją aktualnego stanu gałęzi przez zespół deweloperski.